Position-update mechanisms in the closed loop: a side-by-side derivation of every implemented method

Notes accompanying `closed_loop.position_update`, `closed_loop.soft_loss` and `infl_ens.inflgame.router.verification`

August 2026

Abstract

The closed loop alternates *routing* (assigning prompts to agents), *learning* (LoRA SFT on the assigned prompts) and a *trait-space position update* (a blended step toward a weighted centroid of prompts). The influencer-game theory is only "intact" when the expected position drift points along the strategic gradient $\nabla_{x_i} u_i$. This note writes out, in full, the expected drift of every routing / weighting combination the code implements — hard routing (canonical, strategic, with and without the $(1-G)$ re-weight, expected-pool) and soft top-k routing (the "distributed learning" signal and the "top-k winners at unit loss" ablation) — and shows algebraically which of them are gradient-matched and why the others are not. A single lemma decides every case: the centroid mass must be proportional to $B(b)\,G_i(b)\,(1-G_i(b))$ with a proportionality constant that does not depend on the prompt b. Under soft routing that mass is applied to the *whole* batch, which makes the rule exact for every k and, by Rao–Blackwell, strictly lower-variance than the hard-routing update with the same expectation. Every agent — including a member of a co-located pair — steps independently along its own gradient; merge groups affect only which adapter trains on which prompt, and pairs persist because identical positions receive identical steps, a prediction the run measures rather than enforces. The last section reproduces the numerical cosine and Monte-Carlo checks.

Contents

1 Setting and notation

Trait space and resource. Prompts are embedded into the trait box $[0,1]^L$. A prompt is a point $b \in [0,1]^L$; the prompt distribution is the resource density $B(b)$, normalised so that $\sum_b B(b) = 1$ over a grid (or $\int B\,\mathrm{d}b = 1$). A training round draws a minibatch $b_1, \dots, b_M \overset{\text{iid}}{\sim} B$.

Agents and kernel. There are N agents with positions $x_1, \dots, x_N \in [0,1]^L$, collected in $\mathbf{x}$. Each agent carries a Gaussian kernel with a shared covariance Σ (isotropic, $\Sigma = \sigma^2 I$, unless stated otherwise):

$$f_i(b) \;=\; \exp\Big(-\tfrac{1}{2}\,(x_i - b)^\top \Sigma^{-1} (x_i - b) \Big). \tag{1}$$

Allocation share. The (canonical, proportional) allocation of prompt b to agent i is the softmax over agents,

$$G_i(\mathbf{x}, b) \;=\; \frac{f_i(b)}{\sum_{j=1}^N f_j(b)}, \qquad \sum_{i=1}^N G_i(\mathbf{x}, b) = 1. \tag{2}$$

In code this is `allocation_weights`; every object below is derived from it.

Utility and its gradient. Agent i's utility is the resource it captures,

$$u_i(\mathbf{x}) \;=\; \sum_b B(b)\, G_i(\mathbf{x}, b). \tag{3}$$

Differentiating (2) with respect to x_i only (the other positions are held fixed — this is the *own* gradient that best-response / gradient-ascent dynamics consume):

$$\nabla_{x_i} f_i(b) = f_i(b)\, \Sigma^{-1}(b - x_i), \tag{4}$$

$$\nabla_{x_i} G_i(\mathbf{x}, b) = \frac{\nabla_{x_i} f_i \cdot \sum_j f_j - f_i \cdot \nabla_{x_i} f_i}{(\sum_j f_j)^2} = G_i\,(1 - G_i)\, \Sigma^{-1}(b - x_i), \tag{5}$$

$$\nabla_{x_i} u_i(\mathbf{x}) = \sum_b B(b)\, G_i(\mathbf{x}, b)\, \big(1 - G_i(\mathbf{x}, b)\big)\, \Sigma^{-1}(b - x_i). \tag{6}$$

Equation (6) is `utility_gradient`. The scalar coefficient $G_i(1 - G_i)$ is the whole story: it is *not* G_i, and it is *not* a per-prompt renormalised version of G_i.

Kernel-agnostic form. Nothing above uses the Gaussian shape except the last factor. For any positive differentiable kernel $k(x_i, b)$ with score $s_i(b) := \nabla_{x_i} \log k(x_i, b)$ the same algebra gives $\nabla_{x_i} G_i = G_i(1 - G_i)\, s_i(b)$ and $\nabla_{x_i} u_i = \sum_b B(b) G_i(1 - G_i)\, s_i(b)$ (`kernel_agnostic_gradient_step.tex`); for the Gaussian kernel $s_i(b) = \Sigma^{-1}(b - x_i)$, which is what turns a "score average" into a "centroid minus position". Every statement below is made for the Gaussian case and holds verbatim with $(b - x_i) \to \Sigma\, s_i(b)$ otherwise.

Top-k training support. For a prompt b let $\mathcal{T}_k(b)$ be the set of the k agents with the largest $G_j(\mathbf{x}, b)$ (ties broken by `argpartition`). The *renormalised top-k share* of the update memo is

$$w_i(b) \;=\; \frac{G_i(b)}{Z_k(b)}\,\mathbf{1}\{i \in \mathcal{T}_k(b)\}, \qquad Z_k(b) := \sum_{j \in \mathcal{T}_k(b)} G_j(b), \qquad \sum_i w_i(b) = 1, \tag{7}$$

(`top_k_allocation_weights`). In the kept design the top-k set decides *who trains on what*; it plays no role in the theory-matched position step (Section 4).

2 The common template and the one lemma that decides everything

Every method in the code moves agent i by the same rule: pick a *working set* S_i of prompts and a non-negative *centroid mass* $m_i(b)$ on it, form the mass-weighted centroid, and blend:

$$\hat{x}_i \;=\; \frac{\sum_{b \in S_i} m_i(b)\, b}{\sum_{b \in S_i} m_i(b)}, \qquad x_i \;\leftarrow\; (1-\beta)\, x_i + \beta\, \hat{x}_i, \qquad x_i \;\leftarrow\; \mathrm{clip}(x_i, 0, 1). \tag{8}$$

(`position_from_corpus` / `soft_pair_position_target` with `scores`$= m_i$, then `apply_position_update`; the clip is the `clip_to_box` option.) The displacement is therefore

$$\Delta x_i \;=\; \beta\,(\hat{x}_i - x_i) \;=\; \frac{\beta}{\sum_{b \in S_i} m_i(b)} \underbrace{\sum_{b \in S_i} m_i(b)\,(b - x_i)}_{=:\, d_i}. \tag{9}$$

The prefactor is a positive scalar, so the *direction* of the step is the direction of d_i. Methods differ only in (i) how S_i is chosen (sampled or deterministic) and (ii) what $m_i(b)$ is. When S_i is random, what we can control is the expected mass at each prompt location,

$$\bar{m}_i(b) \;:=\; \mathbb{E}\big[m_i(b)\,\mathbf{1}\{b \in S_i\}\big] \;=\; B(b)\;\Pr[\,b \in S_i\,]\;\cdot\;(\text{weight applied to } b \text{ once it is in } S_i), \tag{10}$$

and the expected direction $\bar{d}_i = \sum_b \bar{m}_i(b)(b - x_i)$.[1]

Lemma 1 (direction matching)**.** *Suppose $\bar{m}_i(b) = c_i\, B(b)\, G_i(b)\,(1 - G_i(b))$ for some constant $c_i > 0$ that does not depend on b. Then*

$$\bar{d}_i \;=\; c_i\, \Sigma\, \nabla_{x_i} u_i(\mathbf{x}),$$

so for isotropic $\Sigma = \sigma^2 I$ the expected step is exactly parallel *to $\nabla_{x_i} u_i$, and for general Σ it is the Σ-preconditioned gradient (still an ascent direction, since $\Sigma \succ 0$).*

Proof. Substitute into $\bar{d}_i = \sum_b \bar{m}_i(b)(b - x_i)$ and compare with (6): $\sum_b c_i B G_i (1 - G_i)(b - x_i) = c_i\, \Sigma \sum_b B G_i (1 - G_i) \Sigma^{-1} (b - x_i) = c_i \Sigma \nabla_{x_i} u_i$. □

Remark 1 (what breaks it). Write $\bar{m}_i(b) = B(b) G_i (1 - G_i)\, \rho_i(b)$. The lemma needs $\rho_i(b) \equiv c_i$. Two distinct ways to violate it appear in the implemented methods:

[1] As in `kernel_agnostic_gradient_step.tex`, $\hat{x}_i$ is a ratio of two Monte-Carlo sums; the law of large numbers in M gives $d_i / \sum m_i \to \bar{d}_i / \sum \bar{m}_i$, and the numerator alone is an unbiased estimator of $\bar{d}_i$. "Matched in expectation" below always refers to $\bar{d}_i$.

1. **Missing factor.** $\rho_i(b) = 1/(1 - G_i(b))$, i.e. the mass is $B\,G_i$ instead of $B\,G_i(1 - G_i)$. The difference $\sum_b B\,G_i^2\,(b - x_i)$ is dominated by the agent's own stronghold (where $G_i \approx 1$), so the step is pulled toward the centre of mass of what the agent already owns rather than toward the contested boundary where the gradient lives.

2. **Per-prompt normaliser.** $\rho_i(b) = 1/Z(b)$ for some $Z(b)$ that varies with b (a softmax over agents evaluated at b, or a top-k renormaliser). Prompts where the normaliser is small are over-weighted relative to the gradient. Renormalising *per agent* (dividing the whole sum by $\sum_b m_i(b)$, as (8) does) is harmless because that constant does not depend on b.

A third potential defect — restricting S_i to a subset of prompts on which $G_i(1-G_i)$ is not identically zero — is a truncation bias. It never appears in the kept design, because the position step always uses either the whole batch or the whole pool (Section 4).

Everything that follows is an application of Lemma 1 and Remark 1 to each method's $\bar{m}_i$.

3 Hard routing: one prompt per (sampled) winning agent

Under `routing_mode: hard` each prompt b_m is assigned to exactly one agent a_m drawn from a categorical distribution $p_\cdot(b_m)$ over agents (`policy: proportional`; `argmax` replaces the draw by $\arg\max_i p_i$). The working set is $S_i = \{m : a_m = i\}$ and a per-prompt centroid weight $\omega_i(b)$ is applied inside it. Hence

$$\Pr[b \in S_i] = p_i(b), \qquad \bar{m}_i(b) \;=\; B(b)\,p_i(b)\,\omega_i(b). \tag{11}$$

The sampling step contributes the factor $p_i(b)$ "for free"; the centroid weight ω_i must supply whatever is missing.

3.1 H1: canonical routing, uniform centroid (`position_update: naive`)

$p_i = G_i$, $\omega_i \equiv 1$:

$$\bar{m}_i(b) = B(b)\,G_i(b), \qquad \bar{d}_i = \sum_b B\,G_i\,(b - x_i) \quad (\textit{not} \text{ matched: missing factor}). \tag{12}$$

This is the historical default (uniform mean of routed embeddings). It is the "canonical naive" column of the verification report. Configs recorded with it are now pinned to `position_update: naive`.

3.2 H2: canonical routing, $(1 - G)$ centroid (`position_update: theory_matched`, the default)

$p_i = G_i$, $\omega_i(b) = 1 - G_i(b)$:

$$\bar{m}_i(b) = B(b)\,G_i(b)\,(1 - G_i(b)), \qquad \bar{d}_i = \Sigma\,\nabla_{x_i} u_i \quad (\text{matched exactly, } c_i = 1). \tag{13}$$

The SFT loss runs at unit weight; only the centroid is re-weighted. This is what the deprecated `loss_reweight: position_only` used to select and is also precisely what the no-SFT simulator `simulate_position_only_loop` realises (`weights_i = 1 - G_batch[i, mine_idx]`). "Position-only" and "theory-matched hard routing" are therefore the same expected dynamics; the former simply skips the LoRA.

3.3 H3: full re-weight (`loss_reweight: one_minus_G`)

Same $\bar{m}_i$ as H2 (matched), but the per-example SFT loss weight is also $s_i(b) = 1 - G_i(b)$. The learning signal then has effective sample size

$$\mathrm{ESS}_i = \frac{\left(\sum_{m \in S_i}(1 - G_i(b_m))\right)^2}{\sum_{m \in S_i}(1 - G_i(b_m))^2} \;\le\; |S_i|,$$

which is small once the agent owns a stronghold ($1 - G_i \approx 0$ on most of its prompts). H2 keeps the full ESS; that is the reason the loss side and the centroid side are decoupled knobs.

3.4 H4: strategic routing (`routing_weight: G_times_1mG`)

$p_i(b) = \dfrac{G_i(1-G_i)}{S(b)}$ with $S(b) := \sum_j G_j(b)(1 - G_j(b))$, $\omega_i \equiv 1$:

$$\bar{m}_i(b) = B(b)\,\frac{G_i(1-G_i)}{S(b)}, \qquad \rho_i(b) = \frac{1}{S(b)} \quad \text{(approximately matched: per-prompt normaliser).} \tag{14}$$

The normaliser $S(b)$ is a genuine function of b (it is large in contested regions and tiny inside strongholds), so by Remark 1(2) the direction is only approximately the gradient — the "strategic" column of the report reads 0.98–0.999 rather than 1.000. Under `theory_matched` the centroid stays *uniform* here: routing already carries the whole coefficient, and applying $(1 - G_i)$ a second time would give $\bar{m}_i \propto B\,G_i(1-G_i)^2/S(b)$, which is why the validator rejects `G_times_1mG` together with any $(1 - G)$ re-weight.

3.5 H5: expected-pool centroid (`centroid_mode: expected_pool`)

Instead of the sampled working set, use the whole pool deterministically with mass $m_i(b_m) = G_i(1 - G_i)$:

$$\hat{x}_i = \frac{\sum_m G_i(b_m)(1 - G_i(b_m))\, b_m}{\sum_m G_i(b_m)(1 - G_i(b_m))}, \tag{15}$$

which is the $M \to \infty$ limit of H2 (`expected_pool_centroid`). Matched by construction; it has no meaningful "naive" variant, so the validator rejects `expected_pool` with `position_update: naive`. It is available under both routing modes.

4 Soft (dense, top-k) routing

Under `routing_mode: soft` nothing is sampled. Each prompt is assigned *deterministically* to all agents in $\mathcal{T}_k(b)$ for *training*:

$$S_i^{\mathrm{train}} = \{m : i \in \mathcal{T}_k(b_m)\}. \tag{16}$$

The crucial difference from Section 3: **there is no sampling factor $p_i(b)$.** Whatever coefficient the gradient needs must be put into the centroid weight ω_i explicitly. The second crucial observation is that the position step need not use S_i^{train} at all: it is a weighted mean of coordinates that are already projected for the whole batch, so using every prompt costs nothing.

4.1 S1: renormalised-share centroid (`position_update: naive`) — the memo's rule

The update memo (steps 1–4) uses the renormalised top-k share (7) as the centroid weight on the training set, $\omega_i(b) = w_i(b)$, $S_i = S_i^{\text{train}}$:

$$\bar{m}_i(b) = B(b)\,\frac{G_i(b)}{Z_k(b)}\,\mathbf{1}\{i \in \mathcal{T}_k(b)\}, \qquad \rho_i(b) = \frac{\mathbf{1}\{i \in \mathcal{T}_k(b)\}}{(1-G_i(b))\,Z_k(b)}. \tag{17}$$

Every defect of Remark 1 occurs at once:

- the $(1-G_i)$ factor is missing — even in the fully dense case $k = N$ (where $Z_N \equiv 1$) the mass is $B\,G_i$, i.e. *exactly the canonical-naive direction H1*;
- for $k < N$ the top-k renormaliser $Z_k(b)$ additionally rescales each prompt by a b-dependent amount, and the indicator truncates the sum.

Writing the dense case out,

$$\bar{d}_i^{\text{S1}} = \sum_b B\,G_i\,(b - x_i) = \underbrace{\sum_b B\,G_i(1-G_i)(b-x_i)}_{\Sigma\nabla_{x_i}u_i} + \underbrace{\sum_b B\,G_i^2\,(b-x_i)}_{\text{stronghold pull}},$$

and the second term can dominate and even reverse the direction: in the verification configuration with one agent per resource mode, the naive cosine is -1.000 while the matched one is $+1.000$ (Section 7). This is the "distributed learning" arm as originally implemented, and it is why the answer to "do the position-only and the distributed-learning steps match in gradient direction?" is *no*. It is kept only as the explicit ablation arm.

4.2 S2: dense $G(1-G)$ mass over the whole batch (`position_update: theory_matched`, the default)

Put the full gradient coefficient into the centroid weight, apply it to *every* prompt of the batch, and do *not* renormalise per prompt:

$$S_i = \{1,\dots,M\}, \qquad m_i(b_m) = G_i(b_m)\,(1-G_i(b_m)), \qquad \bar{m}_i(b) = B(b)\,G_i(b)\,(1-G_i(b)). \tag{18}$$

This is `matched_centroid_mass` applied to the batch allocation matrix. Lemma 1 applies with $c_i = 1$ for *any* `soft_top_k`:

Proposition 1. *Under S2 the expected drift is $\bar{d}_i^{\text{S2}} = \Sigma\,\nabla_{x_i}u_i$ exactly, independently of k. It coincides with the position-only / theory-matched hard dynamics H2 in expectation, and with the expected-pool centroid (15) in the limit $M \to \infty$.*

Proof. The indicator of (17) is gone and $\rho_i \equiv 1$. Since $\Pr[b \in S_i] = 1$, (10) gives $\bar{m}_i(b) = B(b)G_i(1-G_i)$. □

Proposition 2 (Rao–Blackwell: S2 has lower variance than H2)**.** *Fix the batch $b_1,\dots,b_M$. Let $d_i^{\text{H2}} = \sum_m \mathbf{1}\{a_m = i\}\,(1-G_i(b_m))\,(b_m - x_i)$ be the numerator of the hard rule (with the categorical draw $a_m \sim G_\cdot(b_m)$) and $d_i^{\text{S2}} = \sum_m G_i(b_m)(1-G_i(b_m))\,(b_m - x_i)$ that of the dense rule. Then*

$$\mathbb{E}\big[d_i^{\text{H2}} \,\big|\, b_{1:M}\big] = d_i^{\text{S2}}, \qquad \operatorname{Var}\big(d_i^{\text{S2}}\big) \le \operatorname{Var}\big(d_i^{\text{H2}}\big),$$

with strict inequality whenever some $G_i(b_m) \in (0,1)$.

Proof. $\mathbb{E}[\mathbf{1}\{a_m = i\} \mid b_m] = G_i(b_m)$, so conditioning the hard numerator on the batch gives the dense one term by term. The variance statement is the law of total variance,

$$\mathrm{Var}\left(d_i^{\mathrm{H2}}\right) = \mathrm{Var}\left(\mathbb{E}[d_i^{\mathrm{H2}} \mid b_{1:M}]\right) + \mathbb{E}\left[\,\mathrm{Var}(d_i^{\mathrm{H2}} \mid b_{1:M})\right] = \mathrm{Var}\left(d_i^{\mathrm{S2}}\right) + \mathbb{E}\Big[\sum_m G_i(1-G_i)\,(1-G_i)^2\,\|b_m - x_i\|^2\Big],$$

and the second term is positive unless every $G_i(b_m) \in \{0, 1\}$. □

So "just use the hard routing's update" would be correct but wasteful: the dense rule uses all M prompts for every agent instead of the $\approx G_i M$ it happened to win, at the same expectation. The Monte-Carlo table in Section 7 shows the spread roughly halved.

Remark 2 (why the mass must not be renormalised per prompt). Replacing (18) by $B\,G_i(1 - G_i)/Z'(b)$ with $Z'(b) = \sum_j G_j(1 - G_j)$ (or any other b-dependent normaliser) would give $\rho_i(b) = 1/Z'(b)$ and reintroduce Remark 1(2). The per-*agent* normalisation $\sum_m m_i(b_m)$ in (8) is a single positive number and is harmless.

Remark 3 (the training gate is irrelevant to the position step). Because $S_i = \{1, \ldots, M\}$, an agent that trained on no prompt this round (possible for small k) still takes its gradient step; and an agent that trained on every prompt ($k \geq N$) takes exactly the same step it would with $k = 1$. The top-k gate bounds SFT cost; it does not enter the theory.

4.3 S3: co-located merge groups (pairs) — training only

With `sft_merge_groups` a group p of clones shares one LoRA. Define the group share

$$G_p(\mathbf{x}, b) := \sum_{i \in p} G_i(\mathbf{x}, b) \qquad (\texttt{group\_allocation\_weights}). \tag{19}$$

If every group has the same size s and its members are co-located at x_p, then $f_i = f_p$ for $i \in p$ and

$$G_p(\mathbf{x}, b) = \frac{s\, f_p(b)}{\sum_q s\, f_q(b)} = \frac{f_p(b)}{\sum_q f_q(b)},$$

i.e. exactly the allocation of one agent at x_p in the P-player game. That identity is what justifies taking the top-k *training* gate over groups rather than clones.

Positions are not grouped. The group share is used for *nothing else*. Every clone takes its own S2 step in the full N-player game, with its own G_i — its twin appearing as one competitor among the others:

$$m_i(b_m) = G_i(b_m)\big(1 - G_i(b_m)\big) \;\text{ over the whole batch, independently for each } i. \tag{20}$$

No shared step is ever written; the trainer's routing position merely tracks the mean of its members. The reason this is the right design is that the theory already predicts co-location:

Proposition 3 (co-located clones stay co-located, unforced)**.** *If $x_i = x_j$ then $G_i(\mathbf{x}, b) = G_j(\mathbf{x}, b)$ for every b, hence $m_i \equiv m_j$, hence $\hat{x}_i = \hat{x}_j$ and the two clones receive* identical *updates in* (8)*. Co-location is therefore invariant under the independent dynamics: it is a fixed point of the pair's relative coordinate, not a constraint imposed by the update.*

Proof. G_i depends on i only through f_i, which depends only on x_i; equal positions give equal kernels, hence equal shares, masses, targets and blends. □

The practical consequence is that co-location becomes a *measurement* rather than an assumption. Each round logs the within-pair distance $\|x_i - x_j\|$ in `agent_geometry`; if the theory's stability claim were to fail (a partner drifting off under numerical noise, an asymmetric blend cap, a σ above the threshold), the run would show it instead of hiding it behind a shared write. The verification report confirms the mechanism numerically: the twin drifts differ by exactly 0 (Section 7, configuration D).

The historical pair rule survives only as the ablation arm (`position_update: naive`): one step per group, computed from the renormalised group share on the pair's top-k prompts and written to both members. It inherits every defect of S1.

5 The loss side: what the agents actually train on

The learning signal is independent of the centroid mass. With per-example weights $s_i(b)$ the SFT objective is the weight-normalised mean of the response-token cross-entropy,

$$\mathcal{L}_i(\theta_i) = \frac{1}{|S_i^{\text{train}}|} \sum_{m \in S_i^{\text{train}}} s_i(b_m)\, \ell(\theta_i; b_m), \qquad \ell = \text{mean token NLL of the formatted (prompt, response)}, \tag{21}$$

so s_i acts as a per-example learning-rate multiplier (`weighted_causal_lm_loss`); $s_i \equiv 1$ takes the stock unweighted trainer path. The implemented choices are:

routing	config	$s_i(b)$ on the training set
hard	`loss_reweight: null` (default)	1
hard	`loss_reweight: one_minus_G`	$1 - G_i(b)$
soft	`soft_loss: weighted` (default)	$w_i(b) = G_i(b)/Z_k(b)$ ("distributed" signal)
soft	`soft_loss: unit`	1 ("top-k winners" ablation)

With `sft_merge_groups` the group replaces the clone as the unit of this table (G_p, w_p, one adapter per group); the position step is unaffected (Section 4.3).

The top-k winners ablation. Hard routing gives each prompt to *one* winner at unit weight. The ablation `routing_mode: soft`, `soft_loss: unit` gives each prompt to its k highest-share agents, *each at unit weight* — the natural top-k generalisation of one-prompt-per-winning-agent, with $k = 1$ recovering a deterministic (argmax rather than sampled) winner. Compared with the weighted soft signal it removes the share weighting from the learning signal while leaving the assignment untouched; compared with hard routing it replaces one sampled winner by k deterministic ones. Because the centroid mass is chosen separately (and, under `theory_matched`, is the same whole-batch mass for every k), the ablation grid is

$$\{\text{weighted},\ \text{unit}\} \times \{\text{theory\_matched},\ \text{naive}\}\ (\times\ k),$$

and every arm logs `soft_loss`, `position_update`, `soft_top_k`, the batch, and the applied centroid weights (`agent_position_weights`, aligned with `batch_prompts`) per round.

Data volume. Under unit-weight top-k each specialist sees up to $k\times$ the tokens that the de-duplicated pooled replay sees (the replay keeps one copy of each source prompt per round). The comparator is data-matched at the level of source prompts, not of gradient steps; capability comparisons should be read with that asymmetry in mind.

6 Summary table

$\bar{m}_i(b)$ is the expected centroid mass at prompt b (Section 2); "matched" means Lemma 1 holds with a b-independent constant.

method	config keys	$\bar{m}_i(b)/B(b)$	loss weight s_i	matched?
H1 canonical naive	`routing_mode: hard`, `routing_weight: G`, `position_update: naive`	G_i on routed prompts	1	no (missing $1 - G_i$)
H2 theory-matched (default)	`hard`, `G`, `position_update: theory_matched`	$G_i(1 - G_i)$ on routed prompts	1	**exact**
H3 full re-weight	`hard`, `G`, `loss_reweight: one_minus_G`	$G_i(1 - G_i)$ on routed prompts	$1 - G_i$	**exact** (low ESS)
H4 strategic	`hard`, `routing_weight: G_times_1mG`	$G_i(1 - G_i)/S(b)$	1	$\approx$ (per-prompt normaliser)
H5 expected pool	`centroid_mode: expected_pool` (either routing mode)	$G_i(1 - G_i)$ over the pool	any	**exact**
S1 soft naive	`routing_mode: soft`, `position_update: naive`	$G_i \mathbf{1}\{i \in \mathcal{T}_k\}/Z_k(b)$	w_i or 1	no (all defects)
S2 soft matched (default)	`soft`, `position_update: theory_matched`	$G_i(1 - G_i)$ over the whole batch	w_i or 1	**exact**, any k; lower variance than H2
S3 soft pairs	`soft` + `sft_merge_groups`	S2 per clone; G_p used only for the training gate	w_p or 1	**exact** per clone; pairs persist unforced (Prop. 3)

The position-only simulator is H2 without SFT. Hence: *the position-only update and the soft "distributed learning" update agree in expected direction if and only if the soft arm uses S2; the original soft arm (S1) does not, and S2 is additionally the lower-variance estimator of the two.*

7 Numerical verification

`run_reweighted_drift_report` evaluates (6) and each $\bar{d}_i$ on a 25×25 grid over a bimodal resource landscape (modes at $(0.25, 0.25)$ and $(0.75, 0.75)$, width 0.18; kernel $\sigma = 0.20$) and prints the cosine similarity $\cos(\bar{d}_i, \nabla_{x_i} u_i)$. *dense match* is S2 (it does not depend on k); *topk naive* is S1 at the given k. Configuration D is the co-located-pairs check of Section 4.3. The values below are from the run recorded on 2026-08-26.

configuration	agent	strategic (H4)	canon+$(1-G)$ (H2)	canon naive (H1)	dense match (S2)	top3 naive (S1)	top2 naive (S1)
A: near-symmetric Nash	0	0.9994	1.0000	0.4947	**1.0000**	0.4947	0.0887
$x = (.5, .4), (.5, .55), (.45, .5)$	1	0.9935	1.0000	0.9090	**1.0000**	0.9090	0.8718
	2	0.9821	1.0000	0.5587	**1.0000**	0.5587	−0.6336
B: one agent per mode	0	1.0000	1.0000	−1.0000	**1.0000**	−1.0000	−1.0000
+ contested centre	1	1.0000	1.0000	−1.0000	**1.0000**	−1.0000	−1.0000
$x = (.28, .28), (.72, .72), (.5, .5)$	2†	0.0025	−0.0051	0.0023	−0.0072	0.0022	0.0046
C: two on one mode, one alone	0–2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000

D: two co-located clone pairs, four clones, cosines against each *clone's own* $\nabla_{x_i} u_i$ in the 4-player game				
pair	clone	naive pair rule ($k = 2$)	S3 per-clone match	$\|d_{\text{twin}} - d_i\|$
0 at $(0.30, 0.35)$	0	0.9535	**1.0000**	0.00e+00
	1	0.9535	**1.0000**	0.00e+00
1 at $(0.70, 0.60)$	2	0.9749	**1.0000**	0.00e+00
	3	0.9749	**1.0000**	0.00e+00

Reading the table:

- **top3 naive = canon naive**, column for column. This is the identity $Z_N \equiv 1$ in (17): fully dense soft routing with the share centroid is the canonical-naive rule.
- **dense match** = 1.0000 **everywhere the gradient is non-zero**, as Proposition 1 states; it equals the canon+$(1 - G)$ column, i.e. the position-only dynamics, and does so for every k.
- In B the naive direction is *anti*-parallel to the gradient for the two mode-holders: their stronghold pull (17) points back into the mode while the gradient points toward the contested centre.
- †Agent 2 in B sits at a stationary point ($\|\nabla u_2\| \approx 10^{-16}$); its cosines are numerical noise, not evidence for or against any rule.
- In C all drifts are collinear (every rule pushes along the diagonal joining the modes), so the direction test is uninformative there.
- In D the last column is the whole point of Proposition 3: the two members of a pair compute their steps completely independently and the results differ by *exactly* zero, so the pair holds together with nothing enforcing it. Each clone's own drift is parallel to its own gradient; the naive pair rule (one shared step from the renormalised group share) is close but not parallel here — and it is the arm that cannot detect a pair coming apart, since it writes one position to both members by construction.

Finite-batch variance (Proposition 2). The same report draws 200 batches of 4096 prompts, realises each rule on the sampled batch (hard rules with an actual categorical routing draw), and reports the cosine of the *mean* drift with the gradient together with the standard deviation of the drift norm across batches:

configuration	rule	cos(mean drift, ∇u_i)	$\|$mean drift$\|$	std of $\|$drift$\|$
A, agent 0	H2 canonical+$(1-G)$	0.9998	0.0507	0.0052
	S2 dense matched	1.0000	0.0508	**0.0026**
A, agent 1	H2	0.9999	0.0491	0.0050
	S2	1.0000	0.0494	**0.0024**
A, agent 2	H2	0.9999	0.0281	0.0048
	S2	1.0000	0.0282	**0.0025**
B, agent 0	H2	1.0000	0.0916	0.0042
	S2	1.0000	0.0919	**0.0023**
B, agent 1	H2	1.0000	0.0920	0.0043
	S2	1.0000	0.0919	**0.0023**
C, agent 2	H2	1.0000	0.2812	0.0062
	S2	1.0000	0.2814	**0.0034**

The means agree to three decimals (same expectation) and the spread of the dense rule is roughly half that of the hard rule, exactly as the law-of-total-variance decomposition predicts. The naive rules are omitted here; their mean drifts point elsewhere entirely (the cosine table above).

The same facts are asserted by `tests/test_topk_matched.py` (dense matched cosine > 0.9999; naive strictly lower at $k = N$ and $k = 2$; co-located clones take bitwise identical independent steps; dense spread $<$ hard spread) and end-to-end in stubbed closed-loop rounds (hard default applies $(1-G)$; soft default applies the whole-batch $G(1-G)$ mass *per clone* and skips SFT's own position update; `soft_loss: unit` passes no loss weights; partners end each round at the same position with a logged within-pair distance of 0; `expected_pool` works under soft routing).

8 Configuration cheat-sheet

```
closed_loop:
  routing_mode: hard | soft          # sampled single winner | deterministic top-k
  routing_weight: G | G_times_1mG    # canonical | strategic   (hard only)
  soft_top_k: 2                      # soft only; who trains on what
  soft_loss: weighted | unit         # soft only; share-weighted | top-k winners
  loss_reweight: null | one_minus_G  # hard only, loss side; 'position_only'
                                     #   is a deprecated alias for the default
  position_update: theory_matched | naive   # centroid mass (default: matched)
  centroid_mode: batch | expected_pool      # whole batch | whole pool
```

theory_matched applies, per mode, the mass that makes Lemma 1 hold: $(1 - G_i)$ on the routed prompts under hard canonical routing, uniform under strategic routing (the sampling already carries $G_i(1 - G_i)$), and the dense $G_i(1 - G_i)$ over the whole batch under soft routing, independent of `soft_top_k` and of `sft_merge_groups` — every agent steps on its own. **naive** restores the historical centroid (uniform mean under hard routing, renormalised top-k share under soft routing, one shared step per merge group) as an explicit ablation arm.